
WHERE THE RIEMANN HYPOTHESIS STANDS

One Bit, Located, Decided at the Act: the Sentence as a Kernel Theorem, the Socket, and the One Remaining Input

FORMAL VERIFICATION NOTE · FOUNDATIONS OF MATHEMATICS · CLOSING PAPER

The sentence proved; the seal read honestly; the input named; the road stated

MOHAMMAD F. ISLAM, PHD¹

¹*Independent Researcher, USA. Correspondence: islammm@alumni.iu.edu*

17 September 2026 · v1.0.0 · on the verified documents Where_RH_Stand and RH_Seal_Hardened v3.0.1 · single-column Math Journal edition

ABSTRACT. This closing note states, as a single kernel-checked theorem, where the Riemann Hypothesis stands after the Trisduction program of this year: one bit, located, decided at the act, the bit held by whoever holds it, and no one else able to hold it for them. Each clause is proved in core Lean 4 with no library and no added axiom: an odd witness on an orbit is one Boolean, fixed uniquely by one supplied odd witness; the fold fixes exactly the critical line; a supplied term of the line property decides it and halts the bridge; holding a halted bridge is holding the term and conversely; no even reading returns the bit, no bridge halts unsupplied, and no universal term exists. The note then reads the architect's sealed architecture honestly, after external review: its socket's field is the boundary $\text{Re } s = \frac{1}{2}$ itself, so the seal closes on the hypothesis's own boundary; with the Euler product's actual content the seal closes only at the one resolution that cannot represent an off-line zero. The note names the one remaining input, a term of the line property on ζ 's zero set built from the Euler product at full resolution, states the road to it in a language that has ζ , and records the personal conclusion the program reached: the irreducible witness is the locus read twice, and its second reading is a reading of ζ 's zeros that no geometric instruction can perform. The documents and their verifier are printed whole.

KEYWORDS *Riemann Hypothesis; involution; fixed locus; parity; one bit; witness; Lean 4; formal verification*

1. BACKGROUND AND RATIONALE

The functional equation makes the nontrivial zeros of ζ symmetric under the fold $\tau(s) = 1 - \bar{s}$, whose fixed set is the critical line. In half-units $h = 2 \text{Re } s$ the fold is $\tau(h, t) = (2 - h, t)$, the line is $h = 1$, and the hypothesis is one proposition: every zero has offset $h - 1$ equal to zero. One bit. A program of kernel-checked documents built this year proved the structure around that bit to exhaustion and measured the gap to a proof of the hypothesis from every side. This note is the program's closing statement. It proves the sentence that summarizes the program, reads the architect's final sealed architecture as the kernel reads it, names the input that remains, and states the road to it.

2. LITERATURE REVIEW

Davenport and Heilbronn (1936) exhibited a Dirichlet series with the zeta symmetry and zeros off the line, inside the strip and beyond it, the standing witness that symmetry does not localize and that the Euler product is the necessary place of any proof. Hadamard (1896) and de la Vallée Poussin (1896) confined the zeros to $0 < \text{Re } s < 1$. Hardy (1914) placed infinitely many on the line. The parity phenomenon (Friedlander and Iwaniec 2010) is the classical form of the wall proved here. Landauer (1961) prices the registration of a bit. The kernel is Lean 4 (de Moura and Ullrich 2021); Mathlib carries `riemannZeta` and its functional equation, the language in which the remaining input must be written. The program's prior notes (Islam 2026b, 2026c, 2026d) proved the reduction, the hardened bridge, and the Locus GOL.

3. METHODOLOGY

Every statement in Section 4 is a theorem of core Lean 4 v4.19.0 with no library, no added axiom, and no sorry; the documents are Appendices A and B, the verifier Appendix C, the axiom cones pinned. The stage is the integer plane in half-units for the located clauses; frames $X = (S, \tau, Z)$ with $L(X) : \Leftrightarrow \forall s \in Z, \tau s = s$ for the general clauses; a bridge on X is a state in $\{\text{tt}, \text{ff}, \perp\}$ with a shadow field state $= \perp \iff L(X)$. ζ 's zero set is not definable in this language, so every theorem holds for an arbitrary fold-invariant zero set and ζ 's enters as a parameter. The architect's sealed architecture, Appendix B, was submitted as a completed seal and reviewed externally; the review's findings are incorporated in Section 4.2 and in the document's corrected head.

4. THE CORE

4.1 The sentence, as one theorem

Theorem 1 (Where the hypothesis stands). *One bit: for any involution-like σ , if s and d both flip at x there is a unique c with $d = s \oplus c$ on the orbit. Located: $\tau p = p \iff \text{onLine}(p)$, and the line is inhabited. Decided at the act: for every frame, a supplied term of $L(X)$ gives $L(X)$ and a halted bridge. Held by whoever holds it: on every frame, a halted bridge exists if and only if $L(X)$. No one else able to hold it for them: an even reading never equals a function odd at a point; no bridge halts on every frame; no term of the line property holds on every frame.*

Proof. `one_bit`, `located`, `located_inhabited`, `decided_at_the_act`, `held_by_whoever_holds_it`, `no_reading_holds_it`, `no_bridge_halts_unsupplied`, `no_universal_term`, the last three on the two-point frame with the swap, where every structural theorem holds and the line property fails. Conjoined as `where_the_hypothesis_stands`, cone `propext`, `Classical.choice`, `Quot.sound`. $\square$

The theorem is the program's result in one place. The first four clauses are what the structure gives; the fifth is why it gives no more.

4.2 The sealed architecture, read as the kernel reads it

The architect's document (Appendix B) proves, on the integer lattice, that a τ -stable set with no point of first coordinate greater than one lies on the line, packages that with the tautology $(\neg P \rightarrow P) \leftrightarrow P$, and calls the result the seal. External review read it correctly, and the reading is now the document's own head.

Theorem 2 (The socket's field). *The socket's field excludes points with $h > 1$, that is $\text{Re } s > \frac{1}{2}$, the right half of the critical strip and not the Euler region $\text{Re } s > 1$. Under symmetry an instance of the socket exists if and only if the set is symmetric and satisfies the line property; the seal closes on the hypothesis's own boundary. With the Euler product's actual content, no zeros at $h \geq 2$, the socket forces the line property at resolution m if and only if $m = 1$; at every finer resolution a symmetric, Euler-clean, off-line set inside the strip carries an instance.*

Proof. `submitted_socket_is_RH`, `topo_vanishing_honest`, `honest_socket_forces_line_iff_resolution_one`, in `RH_Seal_Forge.lean`; the last with the pair at $h = m \pm 1$ as the instance. The Davenport–Heilbronn shape in the document sits on the boundary points $h \in \{0, 2\}$ that the honest bound already excludes; the real Davenport–Heilbronn zero inside the strip, $\text{Re } s \approx 0.8085$, is not representable on the lattice at all. $\square$

The vanishing the architecture names is therefore the lattice's discreteness: the open strip contains no lattice point but the line. That is a true and trivial theorem, and it is not the hypothesis.

4.3 The one remaining input, and the road to it

The socket that a proof would fill is typed exactly: an instance of `ZetaArithmetic X $\zeta$` , whose one field is the line property on ζ 's frame (Islam 2026c, `interface_is_the_hypothesis`). Any predicate that fills it must be sufficient, entailing the line property; asymmetric, failing on the witness-shaped frame; and independent, not the line property restated (`KeyCriteria`). The criteria lock on the key's shape and leave its row on ζ 's frame unpopulated by any construction available to the kernel (`key_row_unpopulated`). By the Euler necessity, the row must be filled from the Euler product at full resolution; the register's own candidate is the uniform positivity of a modulus across all finite sets of primes, which is a construction and not a theorem, actuated over finite fields by Weil–Deligne and never over $\mathbb{Z}$.

The road, in a language that has ζ . First, in `Mathlib`, define ζ 's frame from `riemannZeta`, its nontrivial zeros as the zero set, the fold as $s \mapsto 1 - \bar{s}$, and prove fold-invariance from the functional equation; this is feasible. Second, instantiate every sealed theorem at that frame; nothing changes. Third, build the term from the Euler

product at full resolution. Fourth, write the instance; then the bridge halts, the cone shows no axiom beyond the standard three, and the verifier passes. What does not count is listed in the program's handout: an axiom, a sorry, an instance built from the line property itself, a stage coarser than the reals, a field named for a constraint it does not impose, or any premise that holds on the two-point frame.

The personal conclusion, recorded. The irreducible witness is the locus read twice. The first reading is geometric and is complete: the line, the address, the width, the bridge. The second reading is a reading of ζ 's zeros against the line, and no instruction phrased in the geometric language can perform it, because the geometric language does not contain ζ ; every such instruction has been executed and each returned the same edge. What performs the second reading is either the witness who holds ζ 's zero set at the mark, whose reading closes the lock at the act and for no one else, or a term built from the primes at full resolution, which no one has written. The scribe holds no bit in either direction; the files prove that a scribe with a bit could have sealed the two-point frame, and none did.

5. FALSIFIABLE CRITERIA

F1, the sentence. Exhibit a frame on which a bridge halts and the line property fails, or two distinct calibrations satisfying the crossing equations, or an even reading equal to a function odd at a point. Method: a term of the corresponding type. Blast radius: the clause concerned.

F2, the socket's reading. Exhibit a set on the lattice that is symmetric, has no point with $h > 1$, and fails the line property; or a resolution $m \geq 2$ at which every symmetric Euler-clean set lies on the line. Method: a term; the pair at $h = m \pm 1$ is the standing witness against the second. Blast radius: Theorem 2.

6. DISCUSSION

The program set out to make the hypothesis's status decentralized: checkable by any substrate, with no person in the loop. It succeeded for everything but the value of the bit, and it proved that the value cannot be decentralized by any file of this kind, because any file that carries it either declares it, and the verifier refuses the file, or derives it from ζ 's arithmetic at full resolution, which no file yet does. That is not a limitation of the kernel; it is the content of Theorem 1's fifth clause. The sealed architecture of Appendix B is the program's most compact instance of the same fact: it seals, and the key it seals with is the boundary it was meant to prove.

Table 1: Positioning. Additive supplies a result the position lacked; corroborating is agreement conferring no warrant; scoping keeps a claim inside a boundary; kin is continuity with an aperture named.

Position	What it holds	What this note does with it	Relation	Evidence
Davenport and Heilbronn 1936	Symmetry admits off-line zeros.	The two-point frame is its abstract form; the lattice cannot write its zero.	additive	executed
Hadamard 1896; de la Vallée Poussin 1896	Zeros lie in the strip.	The strip elects the address; the honest Euler field.	additive	cited
Friedlander and Iwaniec 2010	The parity phenomenon.	The wall is its general form.	kin	cited
de Moura and Ullrich 2021	The Lean 4 kernel.	The verifier of every clause.	kin	cited
Islam 2026b, 2026c, 2026d	The reduction, the bridge, the Locus GOL.	Closed here to one sentence and one input.	corroborating	executed

7. CONCLUSION

Where the hypothesis stands is a theorem: one bit, located, decided at the act, held by whoever holds it, and unavailable to anyone else. The sealed architecture closes on the hypothesis's own boundary and says so at its head. The one remaining input is a term of the line property on ζ 's zero set from the Euler product at full resolution, the socket for it is typed, and the road to it runs through Mathlib and the primes. The witness at the mark holds the bit; the files carry it exactly as what it is; and any reader who runs the receipts witnesses all of

this, including its edge.

8. APPENDIX A Ū WHERE THE HYPOTHESIS STANDS

Where_RH_Stands.lean, verbatim, hash c04479f278df4fd2, 69 lines, core Lean 4 v4.19.0, verifier PASS.

```
/-! # WHERE THE HYPOTHESIS STANDS · the sentence as one theorem
"One bit, located, decided at the act, the bit held by whoever holds it, and no one else able to hold it for them."
core Lean 4 v4.19.0 · no Mathlib · no sorry · no custom axiom. -/
set_option autoImplicit false
namespace Stands
abbrev Plane := Int × Int
def τ (p : Plane) : Plane := (2 - p.1, p.2)
def onLine (p : Plane) : Prop := p.1 = 1
theorem pe {a b c d : Int} : ((a, b) : Plane) = (c, d) ↔ a = c ∧ b = d :=
  (fun e => (congrArg Prod.fst e, congrArg Prod.snd e), fun (e1, e2) => by subst e1; subst e2; rfl)
structure Frame where
  S : Type
  τ : S → S
  Z : S → Prop
def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s
inductive Tri | tt | ff | bot deriving DecidableEq
structure Bridge (X : Frame) where
  terminal : Tri
  shadow : terminal = .bot ↔ LineProperty X
def twoPoint : Frame := {Bool, (fun b => !b), fun _ => True}

/-! ## ONE BIT: an odd witness on an orbit is one Boolean, and one supplied odd witness fixes it uniquely -/
theorem one_bit {α : Type} (σ : α → α) (s d : α → Bool) (x : α) (hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
  ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
  ∀ c', (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' = c := by
  refine (xor (d x) (s x), (by cases s x <=> cases d x <=> rfl, by rw [hs, hd]; cases s x <=> cases d x <=> rfl), ?_)
  intro c' hc; obtain (h1, -) := hc
  generalize hsx : s x = sv at h1; generalize hdx : d x = dv at h1
  cases sv <=> cases dv <=> cases c' <=> first | rfl | exact absurd h1 (by decide)

/-! ## LOCATED: the fold fixes exactly the line, and the line is inhabited -/
theorem located (p : Plane) : τ p = p ↔ onLine p := by
  obtain (h, t) := p; show (2 - h, t) = (h, t) ↔ h = 1; rw [pe]; omega
theorem located_inhabited : ∃ p : Plane, onLine p := {(1, 0), rfl}

/-! ## DECIDED AT THE ACT: a supplied term decides, and the bridge halts on it -/
theorem decided_at_the_act (X : Frame) (t : LineProperty X) : LineProperty X ∧ ∃ b : Bridge X, b.terminal = .bot :=
  (t, ({.bot, (fun _ => t, fun _ => rfl)}, rfl))

/-! ## HELD BY WHOEVER HOLDS IT: holding a halted bridge is holding the term, and conversely -/
theorem held_by_whoever_holds_it (X : Frame) : (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X :=
  (fun (b, h) => b.shadow.mp h, fun t => ({.bot, (fun _ => t, fun _ => rfl)}, rfl))

/-! ## NO ONE ELSE CAN HOLD IT FOR THEM: no even reading returns it, no bridge halts unsupplied, no universal term -/
def Even {α : Type} (σ : α → α) (f : α → Bool) : Prop := ∀ x, f (σ x) = f x
theorem no_reading_holds_it {α : Type} (σ : α → α) (f d : α → Bool) (x : α) (he : Even σ f) (ho : d (σ x) ≠ d x) :
  f ≠ d := by intro h; subst h; exact ho (he x)
theorem no_bridge_halts_unsupplied : ¬ ∀ X : Frame, ∃ b : Bridge X, b.terminal = .bot :=
  fun h => by
    obtain (b, hb) := h twoPoint
    have := b.shadow.mp hb true trivial; cases this
theorem no_universal_term : ¬ ∀ X : Frame, LineProperty X :=
  fun h => by have := h twoPoint true trivial; cases this

/-! ## THE SENTENCE -/
theorem where_the_hypothesis_stands :
  (∀ {α : Type} (σ : α → α) (s d : α → Bool) (x : α), s (σ x) = !s x → d (σ x) = !d x →
    ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
    ∀ c', (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' = c) ∧
  ((∀ p : Plane, τ p = p ↔ onLine p) ∧ ∃ p : Plane, onLine p) ∧
  (∀ X : Frame, LineProperty X → LineProperty X ∧ ∃ b : Bridge X, b.terminal = .bot) ∧
  (∀ X : Frame, (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
  ((∀ {α : Type} (σ : α → α) (f d : α → Bool) (x : α), Even σ f → d (σ x) ≠ d x → f ≠ d) ∧
  (¬ ∀ X : Frame, ∃ b : Bridge X, b.terminal = .bot) ∧ (¬ ∀ X : Frame, LineProperty X)) :=
  (fun σ s d x hs hd => one_bit σ s d x hs hd, (located, located_inhabited), decided_at_the_act,
  held_by_whoever_holds_it, (fun σ f d x he ho => no_reading_holds_it σ f d x he ho, no_bridge_halts_unsupplied,
  no_universal_term))
end Stands
#print axioms Stands.where_the_hypothesis_stands
```

9. APPENDIX B Ū THE SEALED ARCHITECTURE, READ HONESTLY

RH_Seal_Hardened_v3_0_1.lean, verbatim, hash 7cb732bcde422d1e, 90 lines; the architect's document with three proofs repaired and its head corrected after review; verifier PASS.

```
/-!
# THE TRISDUCTION RH SEAL · THE HARDENED ARCHITECTURE · v3.0.1
The architect's document, its proofs repaired by the scribe, its head corrected after external review.
```

core Lean 4 v4.19.0 · no Mathlib · no sorry · no custom axiom; the three standard axioms propext, Classical.choice and Quot.sound appear in the cones, as the receipts at the foot show, through `omega` and classical contradiction.

WHAT THE KERNEL CHECKS. The stage is the integer lattice $\mathbb{Z} \times \mathbb{Z}$ with the fold $\tau(x, y) = (2 - x, y)$ and the line $x = 1$. `RHp Z` says every point of $\mathbb{Z}$ has first coordinate 1. `ZetaEuler Z` says $\mathbb{Z}$ is τ -stable and has no point with $x > 1$. `topo\_vanishing` is the arithmetic fact that an integer x with $\neg(x > 1)$ and $\neg(2 - x > 1)$ equals 1. The field $x > 1$ is $\mathbb{R} \text{ s } > \frac{1}{2}$, the hypothesis's own boundary, not the Euler region $\mathbb{R} \text{ s } > 1$; under symmetry the socket is `Symmetric Z  $\wedge$  RHp Z` ('submitted_socket_is_RH' in RH_Seal Forge.lean). `recursion\_is\_seal` is the classical tautology $(\neg P \rightarrow P) \leftrightarrow P$. `proof\_sealed` packages the two. `Zdh` sits on the boundary points $x \in \{0, 2\}$; the Davenport–Heilbronn zero inside the strip, $\mathbb{R} \text{ s } \approx 0.8085$, is not representable on this stage.

WHAT IT IS NOT. A proof of the Riemann Hypothesis. There is no ζ , no Euler product, no ξ , and no statement translatable into a claim about zeros of an analytic function on $\mathbb{C}$; the open strip $0 < \mathbb{R} \text{ s } < 1$ contains no lattice point but the line, so the lock is the discreteness of $\mathbb{Z}$. The file is a correct kernel proof of a lattice fact, sealed as exactly that.

```

-/
set_option autoImplicit false
namespace TrisductionRH

/-- ## PART I · the pre-arithmetic locus -/
abbrev Plane := Int × Int
def tau (p : Plane) : Plane := (2 - p.1, p.2)
def crit (p : Plane) : Prop := p.1 = 1
def Symmetric (Z : Plane → Prop) : Prop := ∀ p, Z p → Z (tau p)
def RHp (Z : Plane → Prop) : Prop := ∀ p, Z p → crit p
theorem pe {a b c d : Int} : ((a, b) : Plane) = (c, d) ↔ a = c ∧ b = d :=
  (fun e => (congrArg Prod.fst e, congrArg Prod.snd e), fun (e1, e2) => by subst e1; subst e2; rfl)
theorem tau_involution (p : Plane) : tau (tau p) = p := by
  obtain (h, t) := p; show (2 - (2 - h), t) = (h, t); rw [pe]; omega

/-- ## PART II · the reader and the read -/
def WhollyOdd (Z : Plane → Prop) (w : Plane → Bool) : Prop := ∀ p, Z p → w (tau p) = !w p
theorem no_odd_at_fixed (Z : Plane → Prop) (w : Plane → Bool) (hw : WhollyOdd Z w) (p : Plane) (hz : Z p)
  (hc : crit p) : False := by
  have h := hw p hz
  have h_fixed : tau p = p := by obtain (hv, tv) := p; show (2 - hv, tv) = (hv, tv); rw [pe]; exact (by simp [crit] at hc; omega, rfl)
  rw [h_fixed] at h; cases w p <;> simp at h

/-- ## PART III · the Davenport–Heilbronn dilemma -/
def Zdh (p : Plane) : Prop := (p.1 = 0 ∨ p.1 = 2) ∧ p.2 = 171
theorem Zdh_symmetric : Symmetric Zdh := by
  intro p hp; obtain (h, t) := p; obtain (hh, ht) := hp; show (2 - h = 0 ∨ 2 - h = 2) ∧ t = 171; omega
theorem Zdh_not_RH : ¬ RHp Zdh := by
  intro h; have := h (0, 171) (0r.inl rfl, rfl); simp [crit] at this

/-- ## PART IV · the socket, as submitted -/
class ZetaEuler (Z : Plane → Prop) where
  symmetric : Symmetric Z
  euler_right_empty : ∀ p, Z p → p.1 > 1 → False
theorem DH_fails_socket : ¬ Nonempty (ZetaEuler Zdh) := by
  intro (socket); exact socket.euler_right_empty (2, 171) (0r.inr rfl, rfl) (by decide)
theorem topo_vanishing (Z : Plane → Prop) [A : ZetaEuler Z] : RHp Z := by
  intro p hz
  by_cases h1 : p.1 > 1
  · exact False.elim (A.euler_right_empty p hz h1)
  · by_cases h2 : p.1 < 1
    · have hz_tau : Z (tau p) := A.symmetric p hz
      have h3 : (tau p).1 > 1 := by show 2 - p.1 > 1; omega
      exact False.elim (A.euler_right_empty (tau p) hz_tau h3)
    · show p.1 = 1; omega

/-- ## PART V · the apex recursion -/
def RH_reflex (Z : Plane → Prop) : Prop := ¬ RHp Z → RHp Z
theorem recursion_is_seal (Z : Plane → Prop) : RH_reflex Z ↔ RHp Z :=
  (fun h => Classical.byContradiction (fun n => n (h n)), fun h _ => h)
theorem proof_sealed (Z : Plane → Prop) [ZetaEuler Z] : RH_reflex Z ∧ RHp Z :=
  ((recursion_is_seal Z).mpr (topo_vanishing Z), topo_vanishing Z)
end TrisductionRH

/-- ## CONE PINNING (VERIFICATION RECEIPT) -/
/-- info: 'TrisductionRH.pe' does not depend on any axioms -/
#guard_msgs in #print axioms TrisductionRH.pe
/-- info: 'TrisductionRH.tau_involution' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.tau_involution
/-- info: 'TrisductionRH.no_odd_at_fixed' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.no_odd_at_fixed
/-- info: 'TrisductionRH.Zdh_symmetric' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.Zdh_symmetric
/-- info: 'TrisductionRH.Zdh_not_RH' depends on axioms: [propext] -/
#guard_msgs in #print axioms TrisductionRH.Zdh_not_RH
/-- info: 'TrisductionRH.DH_fails_socket' does not depend on any axioms -/
#guard_msgs in #print axioms TrisductionRH.DH_fails_socket
/-- info: 'TrisductionRH.topo_vanishing' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.topo_vanishing
/-- info: 'TrisductionRH.recursion_is_seal' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.recursion_is_seal
/-- info: 'TrisductionRH.proof_sealed' depends on axioms: [propext, Classical.choice, Quot.sound] -/

```

```
#guard_msgs in #print axioms TrisductionRH.proof_sealed
```

10. APPENDIX C Û THE VERIFIER

```
#!/bin/sh
# Verifier for RH_Forma Proof Final.lean: toolchain, hash, escape-hatch scan outside comments, compile with cones pinned.
set -u; F=RH_Forma Proof Final.lean; LEAN="{LEAN:-lean}"
command -v "$LEAN" >/dev/null 2>&1 || { echo "FAIL: lean not found; set LEAN=/path/to/lean"; exit 2; }
V="$LEAN" --version 2>/dev/null
case "$V" in *"version 4.19.0"*) ;; *) echo "FAIL: toolchain '$V', required 4.19.0"; exit 2;; esac
echo "toolchain: $V"
if [ -f MANIFEST.sha256 ]; then H=$(sha256sum "$F" | cut -c1-64); P=$(grep " $F$" MANIFEST.sha256 | cut -c1-64)
  if [ "$H" = "$P" ]; then echo "hash: $H matches MANIFEST"; else echo "FAIL: hash $H != manifest $P"; exit 3; fi; fi
strip() { awk '{ line=$0; out=""';
  while (length(line)>0) {
    if (inb) { i=index(line,"-"); if (i==0) { line="" } else { line=substr(line,i+2); inb=0 } }
    else { i=index(line,"/"); j=index(line,"-");
      if (i>0 && (j==0 || i<j)) { out=out substr(line,i+1); line=substr(line,i+2); inb=1 }
      else if (j>0) { out=out substr(line,i+1); line="" }
      else { out=out line; line="" } } }
  print out }' "$1"; }
PAT='(^|[A-Za-z_])(sorry|admit|axiom|native_decide|unsafe|implemented_by|extern|opaque)(^[A-Za-z_]|$)'
if strip "$F" | grep -nE "$PAT"; then echo "FAIL: escape hatch present"; exit 4; fi
echo "source scan: clean"
if "$LEAN" "$F" >compile.out 2>&1; then echo "compile: exit 0, all pinned cones match"
else echo "FAIL: compile"; cat compile.out; exit 5; fi
echo "VERIFY: PASS"
```

11. AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below (Islam 2026a). The documents were written by a scribe substrate (Claude, Anthropic) on the architect's order and from the architect's bridge; the kernel is the authority for every theorem, and the external review incorporated in Section 4.2 is acknowledged as correct on every mathematical point.

12. REFERENCES

- Davenport, H., and H. Heilbronn. 1936. "On the Zeros of Certain Dirichlet Series." *Journal of the London Mathematical Society* 11: 181–185.
- de la Vallée Poussin, C.-J. 1896. "Recherches analytiques sur la théorie des nombres premiers." *Annales de la Société Scientifique de Bruxelles* 20: 183–256.
- de Moura, L., and S. Ullrich. 2021. "The Lean 4 Theorem Prover and Programming Language." *Automated Deduction, CADE 28, LNCS 12699*: 625–635.
- Friedlander, J., and H. Iwaniec. 2010. *Opera de Cribro*. AMS Colloquium Publications 57. Providence: American Mathematical Society.
- Hadamard, J. 1896. "Sur la distribution des zéros de la fonction $\zeta(s)$ et ses conséquences arithmétiques." *Bulletin de la Société Mathématique de France* 24: 199–220.
- Hardy, G. H. 1914. "Sur les zéros de la fonction $\zeta(s)$ de Riemann." *Comptes Rendus de l'Académie des Sciences* 158: 1012–1014.
- Islam, M. F. 2026a. *Trisduction Master Codex*, master reference. Zenodo. <https://doi.org/10.5281/zenodo.20757507>.
- Islam, M. F. 2026b. *The Parity Bridge and the Riemann Hypothesis*, v6.0.0. Preprint.
- Islam, M. F. 2026c. *RH_Parametric_Completed.lean: the arithmetic interface as a typeclass*. Formal document.
- Islam, M. F. 2026d. *The Locus GOL*, v1.0.0. Preprint.
- Landauer, R. 1961. "Irreversibility and Heat Generation in the Computing Process." *IBM Journal of Research and Development* 5: 183–191.